UPPSALA
UNIVERSITET

Best Practices I: Organising, Debugging and Profiling Python Code

Day 2

Advanced Scientific Programming with Python

Starting Survey

socrative
UU1

How much Python experience do you have?

How much programming experience do you have?

How much time do you spend programming?

Python Versions

Active Python Releases

For more information visit the Python Developer's Guide.

Python version	Maintenance status	First released	End of support	Release schedule
3.9	bugfix	2020-10-05	2025-10	PEP 596
3.8	bugfix	2019-10-14	2024-10	PEP 569
3.7	security	2018-06-27	2023-06-27	PEP 537
3.6	security	2016-12-23	2021-12-23	PEP 494
2.7	end-of-life	2010-07-03	2020-01-01	PEP 373

From https://www.python.org/downloads/ on 2021-02-12

- We'll aim for Python 3.x exclusively (but may sometimes fail…)

Demo: Brief Python Basics Overview

Code Organisation

- Many projects start with a random collection of files

```
icm-39-33:python-course-library filipe$ ls -l
total 2976
-rw-r-----  1 filipe  staff   151355 Apr 25 17:40 1FFK.ipynb
-rw-r-----  1 filipe  staff     3017 Apr 25 17:30 2space_brute_force.py
-rw-r--r--  1 filipe  staff       24 Apr 25 17:25 README.md
-rw-r-----  1 filipe  staff  1360292 Apr 25 17:43 center_of_mass_aliging.ipynb
icm-39-33:python-course-library filipe$
```

- Some Python scripts, some Jupyter notebooks, etc…
- The naming style could be better… (1FFK???)
- As soon as the project grows bigger things quickly get ugly

```
import numpy as np
…

def FSC(F1, F2):
def radial_average(data):
def fun2(z, f1, f2):
def vector_align(cm11, cm12, cm21, cm22):
…
#center of mass f1

cm11 = np.array(ndimage.measurements.center_of_mass(f1))
print(cm11)

cm12 = ndimage.measurements.center_of_mass(f1[:,24:,:])
print(cm12)
cm12 = np.array((14.5, 24, 14.5))
print(cm12)

#center of mass f1

cm21 = np.array(ndimage.measurements.center_of_mass(f2))
print(cm21)

cm22 = ndimage.measurements.center_of_mass(f2[:12,15:,:])
cm22 = np.array((10.299, 16.954, 21.223))
#cm22 = np.array((11.299, 15.954, 22.223))
print(cm22)

angles = vector_align(cm11,cm12,cm21,cm22)
print(angles[0])

f2 = ndimage.interpolation.rotate(f2, angles[3], axes=(0,1), reshape=False, mode='wrap')
f2 = ndimage.interpolation.rotate(f2, angles[1], axes=(1,2), reshape=False, mode='wrap')
```

```
fig = plt.figure()
ax = fig.add_subplot(121)
ax.imshow(np.abs(f1.sum(axis=2)), cmap='plasma')
ax1 = fig.add_subplot(122)
ax1.imshow(np.abs(f2.sum(axis=2)), cmap='plasma')
plt.show()

#fsc = FSC(f1, f2)
#print len(fsc)

fsc_2 = FSC(f1,f2)
# index/pixel in resolution
res = []
for i in range(1,26):
    res.append(1.24E-9/(2*((i*75E-6)/np.sqrt(((i*75E-6)**2+0.4**2)))))

print(res)

res_round = [ '%.1e' % elem for elem in res ]

fig = plt.figure(figsize=((15,5)))
ax = fig.add_subplot(121)
ax.plot(np.arange(len(res)),np.abs(fsc), color='b')
#ax.set_xticklabels(res_round)
ax.set_xlabel('pixels')
ax.axhline(1/np.exp(1), color='r', label='1/e')
ax.axvline(15, color='k', linestyle='-.', label='detector edge')
ax.axvline(21.21, color='k', linestyle='--', label='corner 2D detector')
ax.axvline(25, color='k', label='corner 3D image')
ax.legend(loc='upper right')
ax.set_title('before alignment')
…
```

Code Organisation

- A lot of code outside of functions in the example
- Again ambiguous names (`fun2`)
- Many special values (`10.299, 16.954`, etc…)
- "Sparse" documentation

- Python includes **Classes**, **Modules** and **Packages** to help you organise you code

Python Classes

- Classes are the key features of object-oriented programming.
- A class is a structure for representing an object and the operations that can be performed on the object.
- In Python a class can contain attributes (variables) and methods (functions).
- A class is defined almost like a function, but using the `class` keyword, and the class definition usually contains a number of class method definitions (a function in a class).
- Each class method should have an argument self as its first argument. This object is a self-reference.

Python Classes

- Some class method names have special meaning, for example:
  - `__init__`: The name of the method that is invoked when the object is first created.
  - `__str__` : A method that is invoked when a simple string representation of the class is needed, as for example when printed.
  - There are many more, see https://docs.python.org/3/reference/datamodel.html#special-method-names

```python
class Point:
    """
    Simple class for representing a point in a Cartesian coordinate system.
    """

    def __init__(self, x, y):
        """
        Create a new Point at x, y.
        """
        self.x = x
        self.y = y

    def translate(self, dx, dy):
        """
        Translate the point by dx and dy in the x and y direction.
        """
        self.x += dx
        self.y += dy

    def __str__(self):
        return("Point at [%f, %f]" % (self.x, self.y))
```

Python Classes

- To create a new instance of a class:

```
p1 = Point(0, 0) # this will invoke the __init__ method in the Point class
print(p1)         # this will invoke the __str__ method
```

- To invoke a class method in the class instance p:

```
p2 = Point(1, 1)
p1.translate(0.25, 1.5)
print(p1)
print(p2)
```

- Note that calling class methods can modify the state of that particular class instance, but does not effect other class instances or any global variables.

- That is one of the nice things about object-oriented design: code such as functions and related variables are grouped in separate and independent entities.

Python Modules

- One of the most important concepts in good programming is to reuse code and avoid repetitions.
- Functions and classes with a well-defined purpose and scope
- Reuse instead of repeating similar code in different part of a program (modular programming)
- Greatly improved readability and maintainability
- Fewer bugs, easier to extend and debug/troubleshoot
- Python modules are a higher-level modular programming construct, where we can collect related variables, functions and classes in a module
- A python module is defined in a python file
- Accessible to other Python modules and programs using the `import` statement

```python
"""
Example of a python module. Contains a variable called my_variable,
a function called my_function, and a class called MyClass.
"""

my_variable = 0

def my_function():
    """
    Example function
    """
    return my_variable

class MyClass:
    """
    Example class.
    """

    def __init__(self):
        self.variable = my_variable

    def set_variable(self, new_value):
        """
        Set self.variable to a new value
        """
        self.variable = new_value

    def get_variable(self):
        return self.variable
```

mymodule.py

Python Modules

- We can import the module mymodule into our Python program using import:
  `import mymodule`
- Use `help(module)` to get a summary of what the module provides:
  `help(mymodule)`
- You can also use the built-in `dir()` function to find all the symbols a module expose, e.g. `dir(mymodule)`

Python Modules

```
Help on module mymodule:

NAME
    mymodule

FILE
    /Users/rob/Desktop/scientific-python-lectures/mymodule.py

DESCRIPTION
    Example of a python module. Contains a variable called my_variable,
    a function called my_function, and a class called MyClass.

CLASSES
    MyClass

    class MyClass
     |  Example class.
     |
     |  Methods defined here:
     |
     |  __init__(self)
     |
     |  get_variable(self)
     |
     |  set_variable(self, new_value)
     |      Set self.variable to a new value

FUNCTIONS
    my_function()
        Example function

DATA
    my_variable = 0
```

Python Modules

```
In [1]: import mymodule

In [2]: mymodule.my_variable
Out[2]: 0

In [3]: mymodule.my_function()
Out[3]: 0

In [4]: my_class = mymodule.MyClass()
   ...: my_class.set_variable(10)
   ...: my_class.get_variable()
   ...:
Out[4]: 10
```

```
In [5]: dir(mymodule)
Out[5]:
['MyClass',
 '__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'my_function',
 ‘my_variable']
```

Python Packages

- When you've got a large number of Python Modules, you'll want to organise them into packages.
- Packages are namespaces which contain multiple packages and modules themselves.
- They are simply directories, but with a twist.
- Each package in Python is a directory which **must** contain a special file called `__init__.py`.
- This file can be empty, and it indicates that the directory it contains is a Python package, so it can be imported the same way a module can be imported.

Steps To Create A Python Package

- Working with Python packages is really simple. All you need to do is:
  1. Create a directory and give it your package's name.
  2. Put your classes in it.
  3. Create a `__init__.py` file in the directory
- The `__init__.py` file is executed when the package is imported.
- It is typically used to import classes and modules from the package.

Python Package Example

- In this example, we will create an `animals` package
- It contains two module files named `mammals.py` and `birds.py` with a class each.
- Step 1: Create the Package Directory
- Step 2: Add Modules

```
class Mammals:
    def __init__(self):
        ''' Constructor for this class. '''
        # Create some member animals
        self.members = ['Tiger', 'Elephant', 'Wild Cat']


    def printMembers(self):
        print('Printing members of the Mammals class')
        for member in self.members:
            print('\t%s ' % member)
```

Python Package Example

```
class Birds:
    def __init__(self):
        ''' Constructor for this class. '''
        # Create some member animals
        self.members = ['Sparrow', 'Robin', 'Duck']


    def printMembers(self):
        print('Printing members of the Birds class')
        for member in self.members:
           print('\t%s ' % member)
```

- Step 3: Add the **`__init__.py`** file

```
from .mammals import Mammals
from .birds import Birds
```

- Step 4: Test your package! Create a **`test_animals.py`** file outside of the package

```
# Import classes from your brand new package
from animals import Mammals
from animals import Birds

# Create an object of Mammals class & call a method of it
myMammal = Mammals()
myMammal.printMembers()

# Create an object of Birds class & call a method of it
myBird = Birds()
myBird.printMembers()
```

Coding Style

- It's important to follow a consistent coding style
- Python has its own Style Guide in PEP 8

Python >>> Python Developer's Guide >>> PEP Index >>> PEP 8 -- Style Guide for Python Code

PEP 8 -- Style Guide for Python Code

PEP:	8
Title:	Style Guide for Python Code
Author:	Guido van Rossum <guido at python.org>, Barry Warsaw <barry at python.org>, Nick Coghlan <ncoghlan at gmail.com>
Status:	Active
Type:	Process
Created:	05-Jul-2001
Post-History:	05-Jul-2001, 01-Aug-2013

Coding Style

Google Python Style Guide

▶ Table of Contents

1 Background

Python is the main dynamic language used at Google. This style guide is a list of *dos and don'ts* for Python programs.

To help you format code correctly, we've created a settings file for Vim. For Emacs, the default settings should be fine.

Many teams use the yapf auto-formatter to avoid arguing over formatting.

2 Python Language Rules

2.1 Lint

Run `pylint` over your code using this pylintrc.

2.1.1 Definition

`pylint` is a tool for finding bugs and style problems in Python source code. It finds problems that are typically caught by a compiler for less dynamic languages like C and C++. Because of the dynamic nature of Python, some warnings may be incorrect; however, spurious warnings should be fairly infrequent.

2.1.2 Pros

Catches easy-to-miss errors like typos, using-vars-before-assignment, etc.

- I recommend picking a coding style and sticking to it!

Linters And Formaters

- Linter: Tool that analyzes source code to detect potential errors, unused imports, unused variables, code style violations, and to improve readability.
- Popular linters:
- Autoflake
- Flake8
- Pyflakes
- Pycodestyle
- Pylint
- Ruff

- Formatter: Tool that automatically formats your code to a consistent style, for instance following PEP 8.
- Popular formatters:
- Black
- YAPF
- Ruff

Demo: Ruff Example

Debugging

It is a painful thing
To look at your own trouble and know
That you yourself and no one else has made it

Sophocles, Ajax

- With a debugger, you can:
  - Explore the state of a running program
  - Test implementation code before applying it
  - Follow the program's execution logic
- You can set a **breakpoint** at any point of your program
- Much more powerful than using print() statements everywhere
- Being good at debugging is crucial to become a good programmer
- Python includes the `pdb`, the Python Debugger
- I prefer to use `ipdb`, which is similar but a bit more user friendly

The buggy program

```
$ python main.py
Add the values
It's really that easy
Round
_________________
2, 3, 4, 2, 5
_________________
Sigh. What is your guess?: 16
Sorry that's wrong
The answer is: 5
Like seriously, how could you mess that up
Wins: 0 Loses 1
Would you like to play again?[Y/n]: y
Traceback (most recent call last):
  File "main.py", line 12, in <module>
    main()
  File "main.py", line 8, in main
    GameRunner.run()
  File "/Users/filipe/Documents/Teaching/Advanced Scientific Programming
with Python/python-course/day1-basics/code/pdb-tutorial/dicegame/
runner.py", line 55, in run
    prompt = input("Would you like to play again?[Y/n]: ")
  File "<string>", line 1, in <module>
NameError: name 'y' is not defined
```

ipdb In Action

- First, we have to import `ipdb`
- To analyse the code you need to set a breakpoint, `ipdb.set_trace()`
- Recent python versions allow you to simply do `breakpoint()`
- Lets look at main.py:

```
1 from dicegame.runner import GameRunner
2
3
4 def main():
5     print("Add the values of the dice")
6     print("It's really that easy")
7     print("What are you doing with your life.")
8     GameRunner.run()
9
10
11 if __name__ == "__main__":
12     main()
```

Where you would you put the breakpoint?

ipdb In Action

```python
from dicegame.runner import GameRunner


def main():
    print("Add the values of the dice")
    print("It's really that easy")
    print("What are you doing with your life.")
    import ipdb; ipdb.set_trace() # add pdb here
    GameRunner.run()


if __name__ == "__main__":
    main()
```

- **Let's run main.py again and see what happens.**

```
$ python main.py
Add the values of the dice
It's really that easy
What are you doing with your life.
> /Users/filipe/Documents/Teaching/Advanced Scientific Programming with Python/python-course/day2-bestpractices-1/code/pdb-tutorial/main.py(9)main()
      8     import ipdb; ipdb.set_trace() # add pdb here
----> 9     GameRunner.run()
     10

ipdb>
```

ipdb In Action

- We are now in the middle of the running program and we can start poking around.
- Top (i)pdb commands:
  1. `l(ist)` - Displays 11 lines around the current line or continue the previous listing.
  2. `s(tep)` - Execute the current line, stop at the first possible occasion.
  3. `n(ext)` - Continue execution until the next line in the current function is reached or it returns.
  4. `b(reak)` - Set a breakpoint (depending on the argument provided).
  5. `r(eturn)` - Continue execution until the current function returns.
  6. `w(here)` - Print a stack trace, with the most recent frame at the bottom.

list examples

```
(Pdb) l
  4     def main():
  5         print("Add the values of the dice")
  6         print("It's really that easy")
  7         print("What are you doing with your life.")
  8         import pdb; pdb.set_trace()
  9  ->     GameRunner.run()
 10
 11
 12     if __name__ == "__main__":
 13         main()
[EOF]
```

```
(Pdb) l 1, 13
  1     from dicegame.runner import GameRunner
  2
  3
  4     def main():
  5         print("Add the values of the dice")
  6         print("It's really that easy")
  7         print("What are you doing with your life.")
  8         import pdb; pdb.set_trace()
  9  ->     GameRunner.run()
 10
 11
 12     if __name__ == "__main__":
 13         main()
```

step examples

```
(Pdb) l
  4     def main():
  5         print("Add the values of the dice")
  6         print("It's really that easy")
  7         print("What are you doing with your life.")
  8         import pdb; pdb.set_trace()
  9  ->     GameRunner.run()
 10
 11
 12     if __name__ == "__main__":
 13         main()

(Pdb) s
--Call--
> /Users/Development/pdb-tutorial/dicegame/runner.py(22)run()
-> @classmethod
```

step examples

```
(Pdb) l
 17                 total = 0
 18                 for die in self.dice:
 19                     total += 1
 20                 return total
 21
 22  ->         @classmethod
 23             def run(cls):
 24                 # Probably counts wins or something.
 25                 # Great variable name, 10/10.
 26                 c = 0
 27                 while True:
```

```
(Pdb) s
> /Users/Development/pdb-tutorial/dicegame/runner.py(26)run()
-> c = 0
(Pdb) l
 21
 22             @classmethod
 23             def run(cls):
 24                 # Probably counts wins or something.
 25                 # Great variable name, 10/10.
 26  ->             c = 0
 27                 while True:
 28                     runner = cls()
 29
 30                     print("Round {}\n".format(runner.round))
 31
```

next examples

```
(Pdb) n
> /Users/Development/pdb-tutorial/dicegame/runner.py(27)run()
-> while True:
(Pdb) l
 22         @classmethod
 23         def run(cls):
 24             # Probably counts wins or something.
 25             # Great variable name, 10/10.
 26             c = 0
 27  ->         while True:
 28                 runner = cls()
 29
 30                 print("Round {}\n".format(runner.round))
 31
 32                 for die in runner.dice:
```

next examples

```
(Pdb) n
> /Users/Development/pdb-tutorial/dicegame/runner.py(28)run()
-> runner = cls()
(Pdb) n
> /Users/Development/pdb-tutorial/dicegame/runner.py(30)run()
-> print("Round {}\n".format(runner.round))
(Pdb) n
Round 1

> /Users/Development/pdb-tutorial/dicegame/runner.py(32)run()
-> for die in runner.dice:
(Pdb) l
 27             while True:
 28                 runner = cls()
 29
 30                 print("Round {}\n".format(runner.round))
 31
 32  ->             for die in runner.dice:
 33                     print(die.show())
 34
 35                 guess = input("Sigh. What is your guess?: ")
 36                 guess = int(guess)
```

break examples

- Setting a breakpoint

```
(Pdb) b 35
Breakpoint 1 at /Users/Development/pdb-tutorial/dicegame/runner.py(32)run()
(Pdb) c

[...] # prints some dice

> /Users/Development/pdb-tutorial/dicegame/runner.py(35)run()
-> guess = input("Sigh. What is your guess?: “)
```

- Viewing breakpoints

```
(Pdb) b
Num Type         Disp Enb   Where
1   breakpoint   keep yes   at /Users/Development/pdb-tutorial/dicegame/runner.py:35
    breakpoint already hit 1 time
```

- Clear breakpoints

```
(Pdb) cl 1
Deleted breakpoint 1 at /Users/Development/pdb-tutorial/dicegame/runner.py:35
```

return - I want to get out of this function

r(eturn)

Continue execution until the current function returns.

```
(Pdb) l
 10         def reset(self):
 11             self.round = 1
 12             self.wins = 0
 13             self.loses = 0
 14
 15  ->     def answer(self):
 16             total = 0
 17             for die in self.dice:
 18                 total += 1
 19             return total
 20
(Pdb) r
--Return--
> /Users/Development/pdb-tutorial/dicegame/runner.py(19)answer()->5
-> return total
(Pdb) l
 14
 15         def answer(self):
 16             total = 0
 17             for die in self.dice:
 18                 total += 1
 19  ->         return total
 20
 21         @classmethod
 22         def run(cls):
 23             # Probably counts wins or something.
 24             # Great variable name, 10/10.
(Pdb)
```

where - How did I get here

```
w(here)
   Print a stack trace, with the most recent frame at the bottom.
   An arrow indicates the "current frame", which determines the
   context of most commands.  'bt' is an alias for this command.
```

```
ipdb> w
  /Users/filipe/Documents/Teaching/Advanced Scientific Programming with Python/python-
course/day2-bestpractices-1/code/pdb-tutorial/main.py(13)<module>()
     11
     12 if __name__ == "__main__":
---> 13     main()

  /Users/filipe/Documents/Teaching/Advanced Scientific Programming with Python/python-
course/day2-bestpractices-1/code/pdb-tutorial/main.py(9)main()
      8     import ipdb; ipdb.set_trace() # add pdb here
----> 9     GameRunner.run()
     10

> /Users/filipe/Documents/Teaching/Advanced Scientific Programming with Python/python-
course/day2-bestpractices-1/code/pdb-tutorial/dicegame/runner.py(32)run()
     31             for die in runner.dice:
---> 32                 print(die.show())
     33
```

Arbitrary Python Commands

```
ipdb> l
     27                 runner = cls()
     28
     29                 print("Round {}\n".format(runner.round))
     30
     31                 for die in runner.dice:
---> 32                     print(die.show())
     33
     34                 guess = input("Sigh. What is your guess?: ")
     35                 guess = int(guess)
     36
     37                 if guess == runner.answer():

ipdb> die
<dicegame.die.Die instance at 0x1088b1998>
ipdb> die.value
5
ipdb> die.value = 1
ipdb> die.value
1
ipdb> die.show()
'---------\n|       |\n|   *   |\n|       |\n---------'
ipdb> print(die.show())
---------
|       |
|   *   |
|       |
---------
ipdb>
```

Where Will Execution Stop?

```
1  #!/usr/bin/python
2
3  def fact(x): return (1 if x==0 else x * fact(x-1))
4
5  def is_curious(n):
6      s = str(n)
7      sum = 0;
8      for c in s:
9          sum += fact(int(c))
10     if(sum == n):
11         return True
12     return False
13
14 for a in range(10,1000000):
15     import ipdb; ipdb.set_trace() # add pdb here
16     if(is_curious(a)):
17         print(a)
```

Where will the program stop after you start it?

socrative
UU1

And if you now do `next`?

And if you had done `step` instead of `next`?

And if you now do `continue`?

Demo: Debugging In VSCode

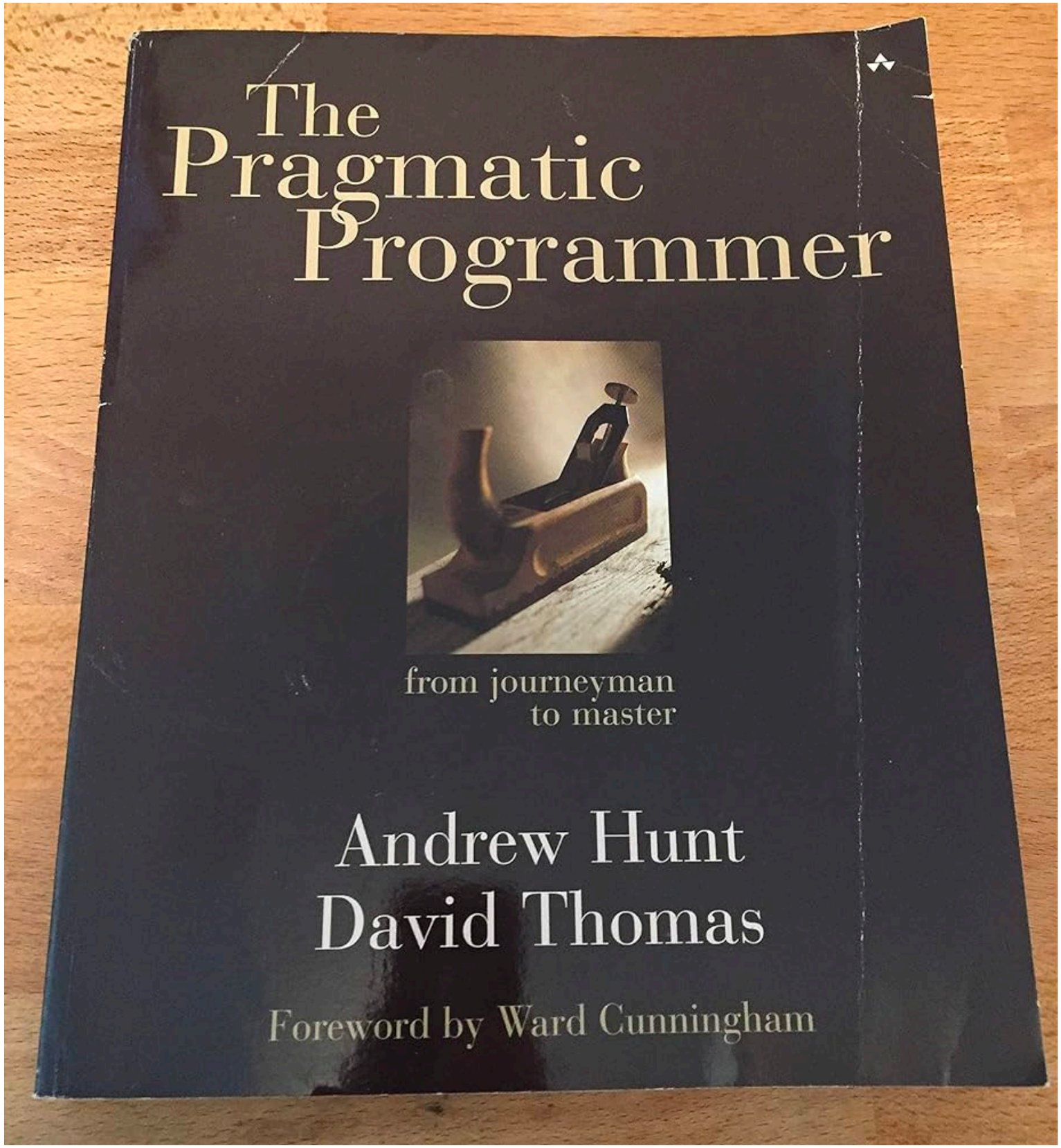
Debugging Tips

- Fix the Problem, not the Blame
- Don't Panic
- Don't Assume It - Prove It

- Is the problem being reported a direct result of the underlying bug, or merely a symptom?
- If you had explained this problem in detail to a coworker, what would you say?
- If the suspect code passes its tests are the tests complete? What happens if you run them with ***this*** data?
- Do the conditions that caused this bug exist anywhere else in the system?

The Pragmatic Programmer

Debugging Tips



Profiling

- Is it worth spending e.g. 2 days to make a program run 20% faster?
- Is it worth optimizing the code so that it spends 90% less memory?
- What does it depend on?

Profiling

- **Measure instead of guessing**
- Before doing code surgery to optimize the run time or lower the memory usage, we should measure where the bottlenecks are. This is called profiling.
- Not only programming beginners can otherwise guess wrong, but also experienced programmers can be surprised by the results of profiling.
- One of the simplest tools is to insert timers

- **CPU profilers:**
- cProfile and profile
- line_profiler
- py-spy
- **Memory profilers:**
- memory_profiler (not actively maintained)
- memray
- **Both CPU and memory:**
- Scalene

Tracing Profilers Vs. Sampling Profilers

Tracing profilers record every function call and event in the program, logging the exact sequence and duration of events.

Pros:

- Provides detailed information on the program's execution.
- Deterministic: Captures exact call sequences and timings.

Cons:

- Higher overhead, slowing down the program.
- Can generate larger amount of data.

Sampling profilers periodically samples the program's state (where it is and how much memory is used), providing a statistical view of where time is spent.

Pros:

- Lower overhead, as it doesn't track every event.
- Scales better with larger programs.

Cons:

- Less precise, potentially missing infrequent or short calls.
- Provides an approximation rather than exact timing.

Tracing Profilers Vs. Sampling Profilers

- Imagine we want to optimize the London Underground system
- We wish to detect bottlenecks in the system to improve the service and for this we have asked few passengers to help us by tracking their journey.
- **Tracing**: We follow every train and passenger, recording every stop and delay. When passengers enter and exit the train, we record the exact time and location.
- **Sampling**: Every 5 minutes the phone notifies the passenger to note down their current location. We then use this information to estimate the most crowded stations and trains.

Choosing The Right System Size

- Sometimes we can configure the system size (for instance the time step in a simulation or the number of time steps or the matrix dimensions) to make the program finish sooner.
- For profiling, we should choose a system size that is representative of the real-world use case. If we profile a program with a small input size, we might not see the same bottlenecks as when running the program with a larger input size.
- Often, when we scale up the system size, or scale the number of processors, new bottlenecks might appear which we didn't see before. This brings us back to: "measure instead of guessing".

Profiling Code

- Python provides the **cProfile** profiler as part of the standard library.
- cProfile is very simple to use, just:
  python -m cProfile script.py
- **Running it on the curious number script:**

```
$ python -m cProfile curious.py
145
40585
         33888831 function calls (6888876 primitive calls) in 3.438 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.062    0.062    3.438    3.438 curious.py:14(find_curious)
        1    0.000    0.000    3.438    3.438 curious.py:3(<module>)
 32888835/5888880    2.610    0.000    2.610    0.000 curious.py:3(fact)
   999990    0.766    0.000    3.375    0.000 curious.py:5(is_curious)
        1    0.000    0.000    3.438    3.438 {built-in method builtins.exec}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.print}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler'
objects}
```

Profiling Code

```python
#!/usr/bin/env python

def fact(x): return (1 if x==0 else x * fact(x-1))

def is_curious(n):
    s = str(n)
    sum = 0;
    for c in s:
        sum += fact(int(c))
    if(sum == n):
        return True
    return False

def find_curious():
    for a in range(10,1000000):
        if(is_curious(a)):
            print(a)

find_curious()
```

```
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.062    0.062    3.438    3.438 curious.py:14(find_curious)
        1    0.000    0.000    3.438    3.438 curious.py:3(<module>)
32888835/5888880    2.610    0.000    2.610    0.000 curious.py:3(fact)
   999990    0.766    0.000    3.375    0.000 curious.py:5(is_curious)
        1    0.000    0.000    3.438    3.438 {built-in method builtins.exec}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.print}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

Line By Line Profiling

- In the previous example we just saw how long each function takes
- Often we want more fine grained knowledge
- The excellent `line_profiler` package provides this
- Install it with `pip install --user line_profiler`
- `line_profiler` comes with the kernprof script to help you run it
- You need to decorate the functions you want to profile with the `@profile` decorator

Line By Line Profiling

```python
#!/usr/bin/python

@profile
def fact(x): return (1 if x==0 else x * fact(x-1))

@profile
def is_curious(n):
    s = str(n)
    sum = 0;
    for c in s:
        sum += fact(int(c))
    if(sum == n):
        return True
    return False

@profile
def find_curious():
    for a in range(10,1000000):
        if(is_curious(a)):
            print a

find_curious()
```

Line By Line Profiling

```
$ kernprof -l -v curious_lineprof.py
145
40585
Wrote profile results to curious_lineprof.py.lprof
Timer unit: 1e-06 s

Total time: 12.4074 s
File: curious_lineprof.py
Function: fact at line 3

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     3                                           @profile
     4  32888835   12407442.0      0.4    100.0  def fact(x): return (1 if x==0 else x * fact(x-1))

Total time: 31.838 s
File: curious_lineprof.py
Function: is_curious at line 6

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     6                                           @profile
     7                                           def is_curious(n):
     8    999990     243710.0      0.2      0.8      s = str(n)
     9    999990     155919.0      0.2      0.5      sum = 0;
    10   6888870    1098314.0      0.2      3.4      for c in s:
    11   5888880   30017594.0      5.1     94.3          sum += fact(int(c))
    12    999990     171378.0      0.2      0.5      if(sum == n):
    13         2          0.0      0.0      0.0          return True
    14    999988     151099.0      0.2      0.5      return False

Total time: 34.9331 s
File: curious_lineprof.py
Function: find_curious at line 16

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    16                                           @profile
    17                                           def find_curious():
    18    999991     150060.0      0.2      0.4      for a in range(10,1000000):
    19    999990   34783007.0     34.8     99.6          if(is_curious(a)):
    20         2          6.0      3.0      0.0              print(a)
```

Times given in microseconds unless noted otherwise.

Profiling In IPython

- You can load line_profiler in IPython with:
  `%load_ext line_profiler`
- Now you'll have access to the magic commands `%lprun` which behave similarly to their command-line counterparts.
- Except you won't need to decorate your functions with the `@profile` decorator.
- But you instead need to tell lprun what function you would like to profile, using the `-f` argument:

```
In [1]: %load_ext line_profiler
In [2]: from primes import primes
In [3]: %lprun -f primes primes(1000)
```

Profiling In IPython

```
In [1]: %load_ext line_profiler
In [2]: from primes import primes
In [3]: %lprun -f primes primes(1000)

Timer unit: 1e-06 s

Total time: 7.7e-05 s
File: /Users/filipe/src/python-course/day2-bestpractices-1/code/primes.py
Function: primes at line 3

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     3                                           def primes(n):
     4         1          2.0      2.0      2.6      if n==2:
     5                                                   return [2]
     6         1          0.0      0.0      0.0      elif n<2:
     7                                                   return []
     8         1          2.0      2.0      2.6      s=list(range(3,n+1,2))
     9         1          5.0      5.0      6.5      mroot = n ** 0.5
    10         1          1.0      1.0      1.3      half=(n+1)/2-1
    11         1          0.0      0.0      0.0      i=0
    12         1          0.0      0.0      0.0      m=3
    13         5          4.0      0.8      5.2      while m <= mroot:
    14         4          4.0      1.0      5.2          if s[i]:
    15         3          1.0      0.3      1.3              j=(m*m-3)//2
    16         3          1.0      0.3      1.3              s[j]=0
    17        31         15.0      0.5     19.5              while j<half:
    18        28         12.0      0.4     15.6                  s[j]=0
    19        28         13.0      0.5     16.9                  j+=m
    20         4          4.0      1.0      5.2          i=i+1
    21         4          4.0      1.0      5.2          m=2*i+3
    22         1          9.0      9.0     11.7      return [2]+[x for x in s if x]
```

Profiling In IPython

- As a simpler profiling alternative you can use **%time**, which does a single measurement of a function:

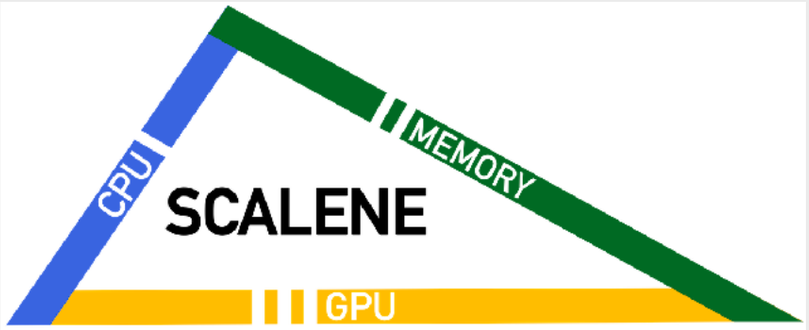
```
In [5]: %time primes(100)
CPU times: user 16 µs, sys: 0 ns, total: 16 µs
Wall time: 18.1 µs
Out[5]:
[2,
 3,
...
 89,
 97]
```

- And **%timeit**, which does repeated measurements to arrive at a more statistically significant result:

```
In [6]: %timeit primes(100)
2.58 µs ± 12.2 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

- This can save you a lot of time and effort since none of your source code needs to be modified in order to use these profiling commands.

Demo: Profiling With Scalene



https://github.com/plasma-umass/scalene

Demo: Memory Profiling With Memray

https://github.com/bloomberg/memray

Final Thoughts On Profiling

- Timing your code will change its timing
- Profiling incurs some performance penalty
- The finer the profiling the greater the penalty
- 10s with `cProfile` vs 70s with `line_profiler`
- **Always profile before optimising**

"We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%."

Donald Knuth

References

Code examples have been take from

- https://github.com/jrjohansson/scientific-python-lectures/blob/master/Lecture-1-Introduction-to-Python-Programming.ipynb
- http://pythoncentral.io/how-to-create-a-python-package/
- https://www.learnpython.org/en/Modules_and_Packages
- https://github.com/spiside/pdb-tutorial
- http://lanyrd.com/2013/pycon/scdywg/
- https://www.huyng.com/posts/python-performance-analysis
- https://aaltoscicomp.github.io/python-for-scicomp/profiling/